



IFET **Autonomous** College of Engineering

Permanently Affiliated to Anna University | Approved by AICTE

Water Leakage & Wastage Prediction Using Machine Learning

Water is one of the most essential natural resources required for daily life, agriculture, industries, and various human activities. Water leakage and excessive water usage are major problems that lead to the wastage of valuable water resources. Detecting water leakage and predicting water wastage manually can be difficult and time-consuming, especially when abnormal water flow occurs without immediate observation. This project, “Water Leakage & Wastage Prediction Using Machine Learning,” aims to develop an automated system that predicts water usage conditions based on important parameters such as flow rate, usage duration, water pressure, flow variation, and daily water usage. The system uses these parameters as input features and applies a Random Forest machine learning model to classify the water condition as Normal, Wastage, or Leakage. The model is trained using a dataset containing different water usage conditions and evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. Based on the predicted water condition, the system provides an appropriate alert message to help identify possible leakage and excessive water wastage. The developed system demonstrates the practical application of Machine Learning in water usage monitoring, leakage identification, and water conservation.

PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Create the Dataset

```
np.random.seed(42)
n = 1000
data = {
    "Flow_Rate": np.random.uniform(1, 15, n),
    "Usage_Duration": np.random.uniform(1, 60, n),
    "Water_Pressure": np.random.uniform(1, 10, n),
    "Flow_Variation": np.random.uniform(0, 8, n),
    "Daily_Usage": np.random.uniform(10, 200, n)
}
df = pd.DataFrame(data)
# Calculate a water condition score
score = (
    (df["Flow_Rate"] > 10).astype(int)
    + (df["Usage_Duration"] > 40).astype(int)
    + (df["Flow_Variation"] > 5).astype(int)
    + (df["Daily_Usage"] > 140).astype(int)
)
# Create target labels
df["Water_Status"] = np.where(
    score >= 3,
    "Leakage",
```

```

np.where(
    score >= 1,
    "Wastage",
    "Normal"
)
)
df.head()

```

OUTPUT:

	Flow_Rate	Usage_Duration	Water_Pressure	Flow_Variation	Daily_Usage	Water_Status
0	6.243562	11.922843	3.355351	5.381624	118.679217	Wastage
1	14.310000	32.972156	3.222809	6.373451	163.032143	Leakage
2	11.247915	52.503804	9.156291	2.003743	154.430577	Leakage
3	9.381219	44.201268	3.245916	4.998993	39.240982	Wastage
4	3.184261	48.587108	3.447548	4.573968	38.357399	Wastage

Check the Dataset

```

print("Dataset Shape:", df.shape)
print("\nDataset Information:")
print(df.info())
print("\nMissing Values:")
print(df.isnull().sum())
print("\nWater Status Distribution:")
print(df["Water_Status"].value_counts())

```

OUTPUT:

Dataset Shape: (1000, 6)

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Flow_Rate	1000 non-null	float64
1	Usage_Duration	1000 non-null	float64
2	Water_Pressure	1000 non-null	float64
3	Flow_Variation	1000 non-null	float64
4	Daily_Usage	1000 non-null	float64
5	Water_Status	1000 non-null	object

dtypes: float64(5), object(1)

memory usage: 47.0+ KB

None

Missing Values:

Flow_Rate	0
Usage_Duration	0
Water_Pressure	0
Flow_Variation	0
Daily_Usage	0
Water_Status	0

dtype: int64

Water Status Distribution:

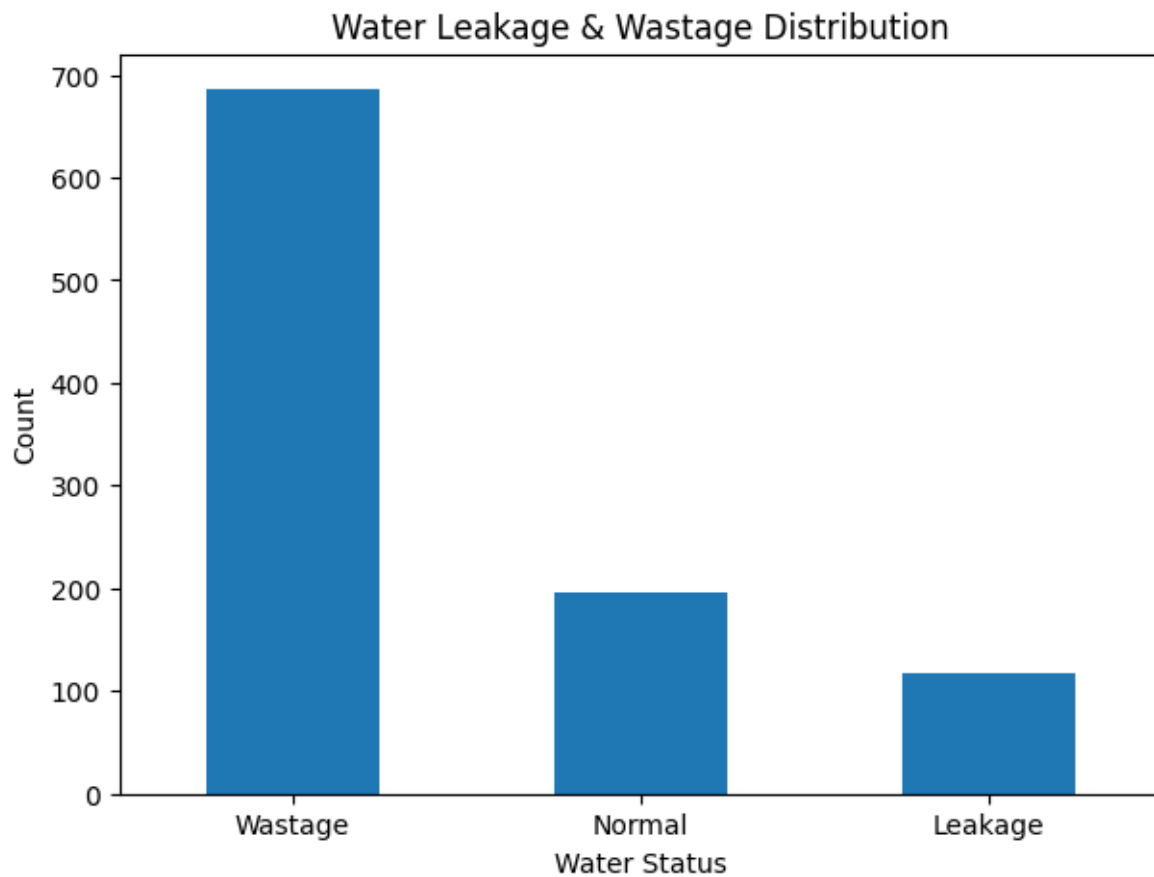
Water_Status	
Wastage	686
Normal	196
Leakage	118

Name: count, dtype: int64

Visualize Water Status Data

```
df["Water_Status"].value_counts().plot(kind="bar")  
plt.xlabel("Water Status")  
plt.ylabel("Count")  
plt.title("Water Leakage & Wastage Distribution")  
plt.xticks(rotation=0)  
plt.show()
```

OUTPUT:



Separate Input and Output

```
X = df.drop("Water_Status", axis=1)  
y = df["Water_Status"]  
print("Input Features:")  
print(X.head())  
print("\nTarget Values:")  
print(y.head())
```

OUTPUT:

Input Features:

	Flow_Rate	Usage_Duration	Water_Pressure	Flow_Variation	Daily_Usage
0	6.243562	11.922843	3.355351	5.381624	118.679217
1	14.310000	32.972156	3.222809	6.373451	163.032143
2	11.247915	52.503804	9.156291	2.003743	154.430577
3	9.381219	44.201268	3.245916	4.998993	39.240982
4	3.184261	48.587108	3.447548	4.573968	38.357399

Target Values:

0	Wastage
1	Leakage
2	Leakage
3	Wastage
4	Wastage

Name: Water_Status, dtype: object

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)  
  
print("Training Data:", X_train.shape)  
print("Testing Data:", X_test.shape)
```

OUTPUT:

Training Data: (800, 5)

Testing Data: (200, 5)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("Feature Scaling Completed")
```

OUTPUT:

Feature Scaling Completed

Train Random Forest Model

```
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
model.fit(X_train_scaled, y_train)
print("Model Training Completed Successfully")
```

OUTPUT:

Model Training Completed Successfully

Make Predictions

```
y_pred = model.predict(X_test_scaled)
print("Prediction Completed")
print(y_pred[:10])
```

OUTPUT:

Prediction Completed

```
['Wastage' 'Wastage' 'Wastage' 'Wastage' 'Wastage' 'Leakage' 'Wastage'
 'Normal' 'Wastage' 'Wastage']
```

Calculate Model Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", round(accuracy * 100, 2), "%")
```

OUTPUT:

Model Accuracy: 98.0 %

Classification Report

```
print("Classification Report:\n") print( classification_report( y_test, y_pred, zero_division=0 ) )
```

OUTPUT:

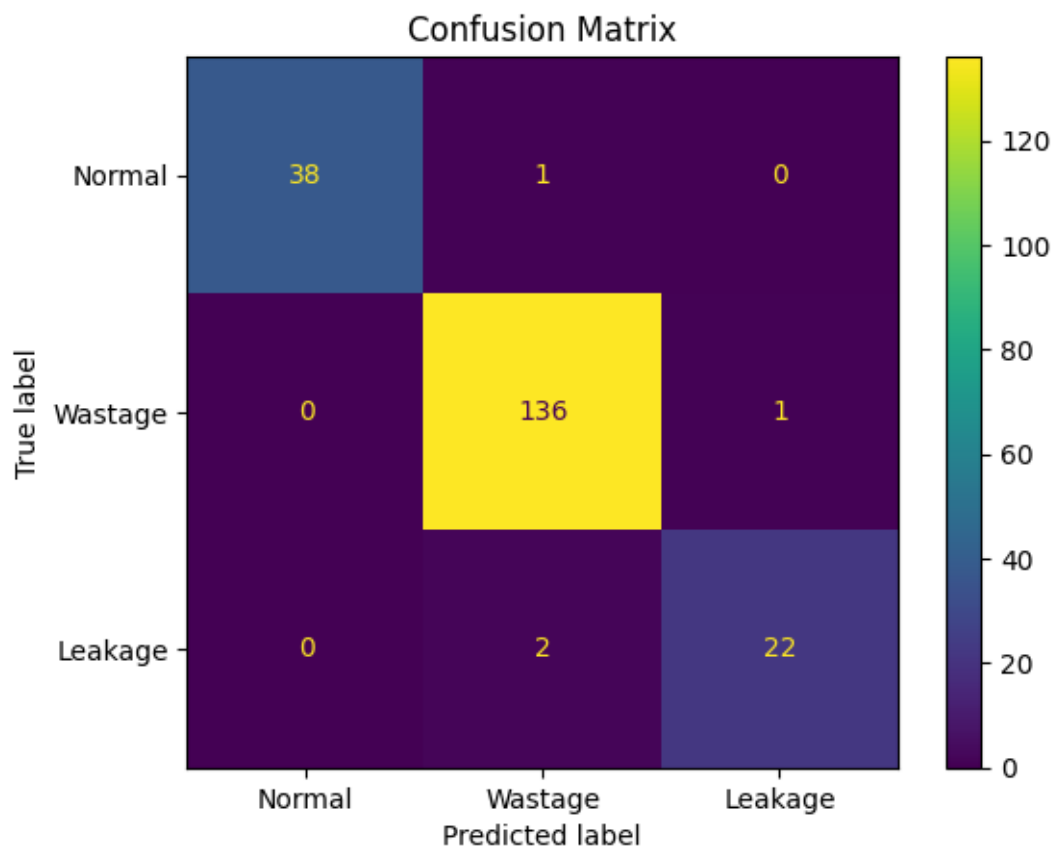
Classification Report:

	precision	recall	f1-score	support
Leakage	0.96	0.92	0.94	24
Normal	1.00	0.97	0.99	39
Wastage	0.98	0.99	0.99	137
accuracy			0.98	200
macro avg	0.98	0.96	0.97	200
weighted avg	0.98	0.98	0.98	200

Confusion Matrix

```
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=["Normal", "Wastage", "Leakage"]
)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Normal", "Wastage", "Leakage"]
)
disp.plot()
plt.title("Confusion Matrix")
plt.show()
```

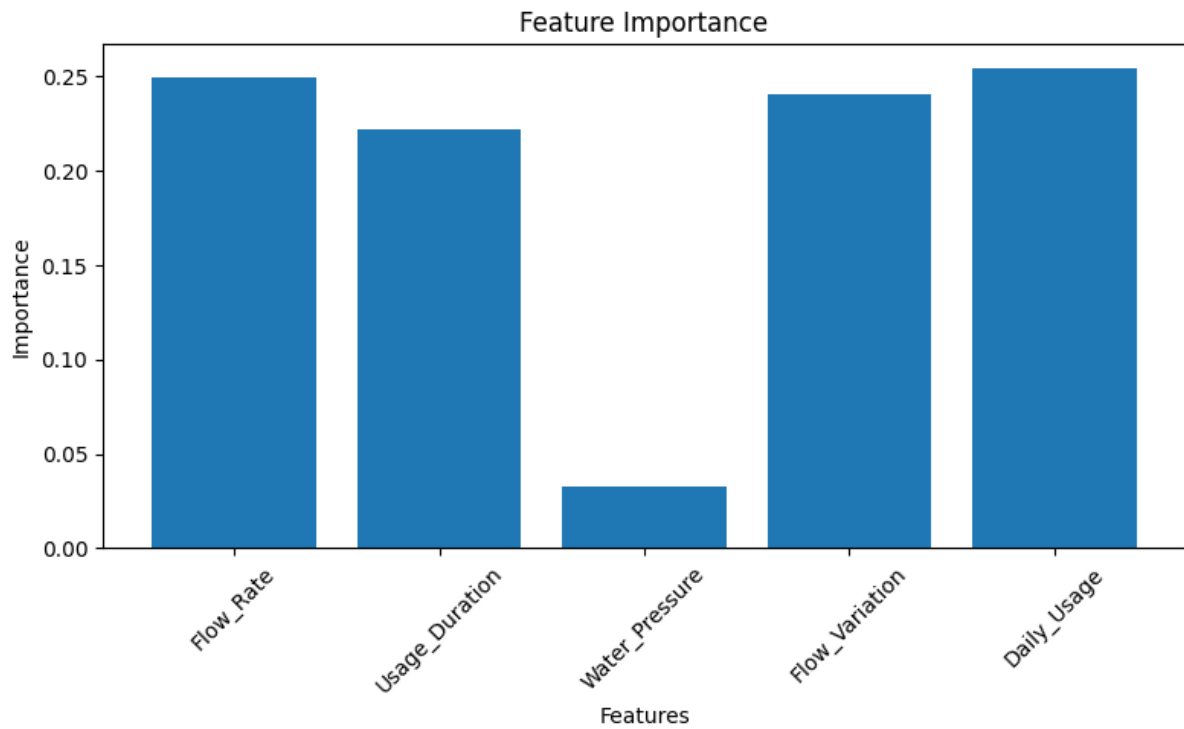
OUTPUT:



Feature Importance Graph

```
importance = model.feature_importances_  
features = X.columns  
plt.figure(figsize=(8, 5))  
plt.bar(features, importance)  
plt.title("Feature Importance")  
plt.xlabel("Features")  
plt.ylabel("Importance")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

OUTPUT:



Test New Water Usage Data

```
def predict_water_status(  
    flow_rate,  
    usage_duration,  
    water_pressure,  
    flow_variation,  
    daily_usage  
):  
    new_data = pd.DataFrame({  
        "Flow_Rate": [flow_rate],  
        "Usage_Duration": [usage_duration],  
        "Water_Pressure": [water_pressure],  
        "Flow_Variation": [flow_variation],  
        "Daily_Usage": [daily_usage]  
    })
```

```
new_data_scaled = scaler.transform(new_data)
prediction = model.predict(new_data_scaled)
print("Predicted Water Status:", prediction[0])
if prediction[0] == "Leakage":
    print("Alert: Possible Water Leakage")
elif prediction[0] == "Wastage":
    print("Alert: Possible Water Wastage")
else:
    print("Status: Normal Water Usage")
```

Run Test

```
predict_water_status(
    flow_rate=12,
    usage_duration=50,
    water_pressure=7,
    flow_variation=6,
    daily_usage=170
)
```

OUTPUT:

Predicted Water Status: Leakage

Alert: Possible Water Leakage